Journey Booking System

Technical Architecture Report

Module: CS7NS6 Distributed Systems — Exercise 2

Date: March 2026

Name	Service Responsibility
Parth Deshmukh	User Service
Tanmay Ghosh	Journey Service
Sneha Meto	Conflict Detection Service
Aditya Kumar Singh	Notification Service
Saurabh Deshmukh	Enforcement Service
Sai Eeshwar Divaakar	Analytics & Monitoring Service

Contents

1 Introduction **1**

2 System Requirements **1**
2.1 Functional Requirements 2
2.2 Non-Functional Requirements 3
2.3 Failure Model 3

3 Technical Architecture **3**
3.1 High-Level Overview 3
3.2 Service Decomposition 4

4 Technology Stack **5**

5 Summary **5**

1 Introduction

This report outlines the technical architecture of a globally-accessible journey pre-booking system for road-vehicle drivers. The system allows drivers to book, cancel, and manage journeys, receive notifications of booking status, and be verified by enforcement agents. It is designed to serve millions of users worldwide with appropriate levels of performance, scalability, availability, and reliability.

The system is structured as **six distributed microservices** operating in a loosely-coupled manner, communicating via synchronous REST for request-response flows and asynchronous event passing through a message broker for decoupled inter-service communication.

The system comprises six loosely-coupled microservices communicating via synchronous REST for request-response flows and asynchronous event passing through a message broker for decoupled inter-service communication.

2 System Requirements

2.1 Functional Requirements

Table 1: Functional Requirements

ID	Requirement	Description
FR1	User Registration & Authentication	Drivers register an account, authenticate via JWT, and manage their profile including driving licence and vehicle registration details. Passwords are hashed with bcrypt.
FR2	Journey Booking	Drivers request a journey by specifying origin, destination, departure time, estimated duration, and vehicle. The system confirms or rejects the booking based on conflict checks.
FR3	Conflict Detection	The system checks for scheduling conflicts — a driver or vehicle cannot have overlapping journeys — and verifies that road capacity is not exceeded in any geographic area.
FR4	Journey Cancellation	Drivers may cancel a previously confirmed journey. Cancellation propagates to all downstream services via event messages.
FR5	Real-Time Notifications	Drivers are notified of booking confirmations, rejections, and cancellations via WebSocket push and stored notification history.
FR6	Enforcement Verification	Enforcement agents verify, by vehicle registration or driving licence number, whether a driver currently holds a valid, active journey booking.
FR7	Analytics & Monitoring	System operators view real-time statistics (bookings per day, rejection rates, active journeys) and monitor the health of all services via a dashboard.

2.2 Non-Functional Requirements

Table 2: Non-Functional Requirements

Property	Target	Rationale
Availability	99.5% (≈43 h downtime/year)	Single-instance services with `restart: unless-stopped`; no replica load balancing in current deployment — a container restart or brief DB outage creates a gap.
Booking Latency	< 2 s (p95)	Happy path (DB write → sync conflict check → DB update → RabbitMQ publish) benchmarks at ∼100–500 ms under light load; 2 s accommodates p95 under concurrent requests.
Verification Latency	< 500 ms (p95)	Redis cache hit is sub-millisecond; even the fallback REST path stays well under 500 ms in a co-located deployment.
Peak Throughput	∼500 bookings/s	Each service runs two Uvicorn workers; the conflict service is the serial bottleneck — horizontal scaling would raise this proportionally.
Consistency	Strong per-user writes; eventually consistent global reads	A driver always sees their own bookings immediately.
Durability	No confirmed booking lost	Zero tolerance for data loss of confirmed bookings.

2.3 Failure Model

We adopt a **crash-recovery** failure model:

- **Process failures:** Services may crash at any time and restart. All persistent state is stored in durable databases, so no committed data is lost on crash.
- **Network failures:** Messages between services may be delayed, reordered, or lost. We handle this through retries, idempotency keys, and persistent message queues.
- **Byzantine faults:** We assume a trusted internal network between services. Byzantine fault tolerance is not required and would add unjustified complexity.
- **Correlated failures:** Mitigated by deploying service replicas independently (in a production setting, across availability zones).

3 Technical Architecture

3.1 High-Level Overview

The system follows a **microservices architecture** with an API Gateway fronting all client traffic. Services communicate synchronously (REST over HTTP) for request-response flows and asynchronously (RabbitMQ message broker) for event-driven flows.

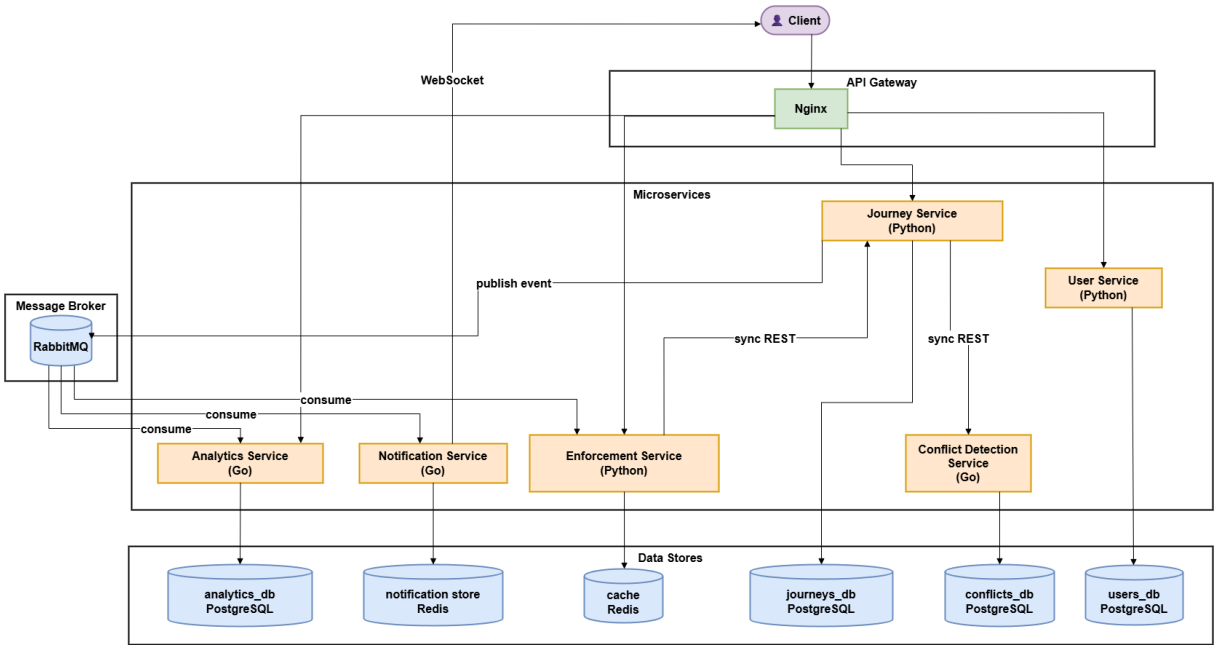


Figure 1: Architecture diagram

3.2 Service Decomposition

Each of the six services maps to one group member and encapsulates a distinct bounded context:

#	Service	Responsibility	Database	Stateful?
1	User Service	Registration, authentication (JWT), driver profiles	PostgreSQL (users_db)	Yes
2	Journey Service	Journey CRUD, booking saga orchestration, idempotency	PostgreSQL (journeys_db)	Yes
3	Conflict Detection	Time-overlap detection, vehicle-overlap, road-capacity checking	PostgreSQL (conflicts_db)	Yes
4	Notification Service	Consume events and deliver notifications (WebSocket, stored history)	Redis (notification store)	Minimal
5	Enforcement Service	Verify active bookings by vehicle or licence number	Redis (cache) + Journey Svc	Minimal
6	Analytics Service	Event logging, real-time counters, aggregated stats, health dashboard	PostgreSQL (analytics_db)	Yes

Table 3: Service Decomposition

Each stateful service owns its own database instance (**database-per-service** pattern), ensuring loose coupling and independent deployability.

4 Technology Stack

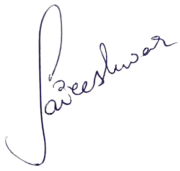
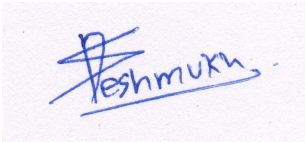
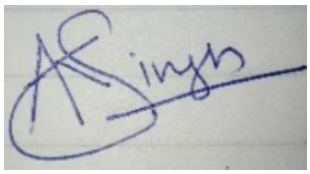
Component	Technology	Justification
Language	Python 3.12	Fast development cycle, rich async ecosystem, group familiarity. Suitable for User, Journey, and Enforcement Services.
	Go 1.22	Offers faster execution and reduced latency. Ideal for high-throughput microservices, native goroutine concurrency, and minimal memory footprint in distributed components. Can be utilised for Conflict Detection, Notification and Analytics Services.
Web Framework	FastAPI	Native async/await, high throughput, automatic OpenAPI documentation.
Database	PostgreSQL 16	ACID guarantees, mature streaming replication, table partitioning, JSON support.
Cache	Redis 7	Sub-millisecond lookups, TTL-based expiry, append-only persistence.
Message Broker	RabbitMQ 3.13	Durable topic-based routing, dead-letter queues, built-in management UI.
API Gateway	Nginx	Industry-standard reverse proxy, built-in rate limiting and health checks.
ORM	SQLAlchemy 2.0 (async)	Mature ORM with full async support and migration tooling (Alembic).
Authentication	JWT (PyJWT + bcrypt)	Stateless, scalable, no session store required.
Containerisation	Docker + Docker Compose	Reproducible multi-service deployment, straightforward failure injection for testing.

Table 4: Technology Stack

5 Summary

Our system decomposes the journey booking problem into six independently deployable microservices, each owned by one group member. The architecture applies several distributed systems techniques like the saga pattern, asynchronous messaging, caching with fallback, idempotency, rate limiting, and geographic partitioning, all while deliberately avoiding unnecessary complexity such as Byzantine fault tolerance or distributed locking. The result is a system that balances performance, scalability, and reliability.

Signatures

Name	Signature	Date
Sneha Meto		10/3/2026
Sai Eeshwar Divaakar		10/3/2026
Saurabh Deshmukh		10/3/2026
Parth Deshmukh		10/3/2026
Aditya Kumar Singh		10/3/2026
Tanmay Ghosh		10/3/2026